PolicyBot

An AI assistant that answers questions about company policies, and always shows where the answer came from

A step-by-step account of how I built it and how well it works

Project: AI Engineering Project (Retrieval-Augmented Generation)
Code: https://github.com/Valuoch/rag-policy-ai
Date: 12 October 2026
Reader: no technical background needed. Technical words are explained when they first appear.

The short version

I built a chat assistant, called **PolicyBot**, for a made-up company named **Lumora Systems**. An employee types a question such as *"How many vacation days do I get?"* or *"What's the hotel limit in New York?"*, and PolicyBot answers in a few sentences. Each answer comes with numbered **sources**: the exact policy, section and sentence it relied on, with a link to read it in full.

PolicyBot does not answer from general knowledge. It answers **only** from the company's own policy documents, and if the answer is not in them, it says so instead of guessing.

What I measured	Result	What it means
Answers backed by the documents	92.6%	Answers contain nothing made up
Sources pointing to the right passage	100%	The links really support the answer
Answers containing the key facts	100%	Compared with answers written in advance
Off-topic questions politely declined	100%	It doesn't wander outside its job
Typical time to answer	1.2 seconds	95% of answers within 1.7 seconds

Results from the AI-written mode, on a test of 30 questions. Step 12 explains how these were measured.

Everything required by the assignment was delivered:

- A library of 13 realistic company policies (about 67 pages).
- A system that reads, organises and searches those policies by meaning.
- An AI that writes short answers with sources, plus safety rules.
- A web page to chat with it, and a health-check page.
- Automatic quality checks that run every time the code changes, with a report.
- A measured evaluation of accuracy and speed, and written design documents.
- All code published at https://github.com/Valuoch/rag-policy-ai.

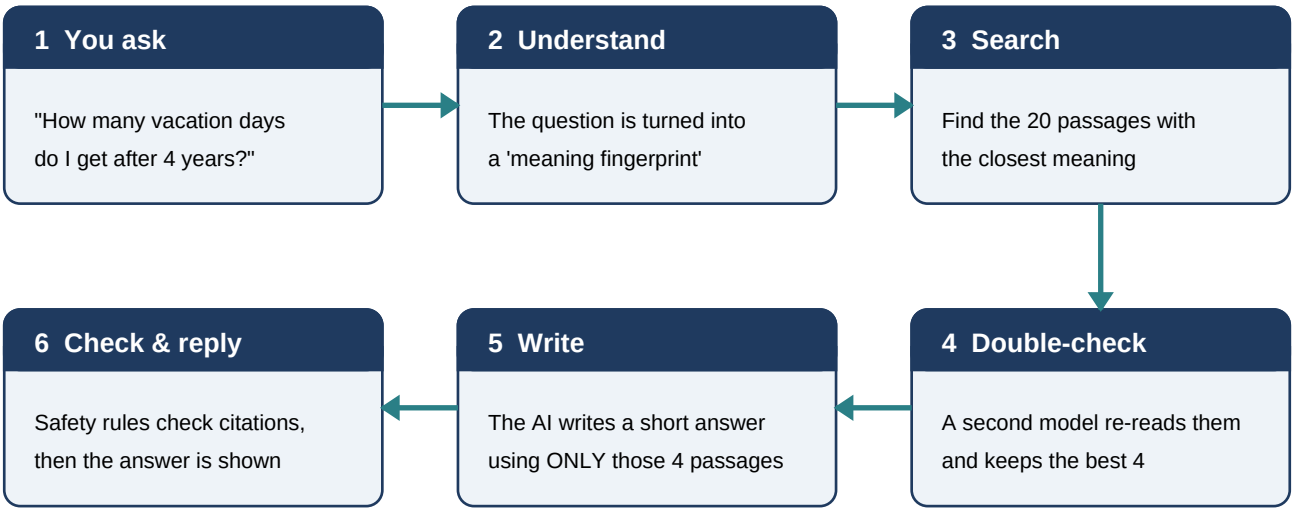
A few ideas in plain English

You don't need any of these terms to follow the report, but they appear in the code and documents.

Term	Plain-English meaning	Everyday comparison
Large language model (LLM)	An AI that reads and writes text, like the one behind ChatGPT.	A very well-read writer who can be confidently wrong.
RAG (Retrieval-Augmented Generation)	First **find** the relevant pages, then let the AI **write** an answer using only those pages.	An **open-book exam** instead of answering from memory.
Document chunk	A short passage (about half a page) cut from a policy.	An index card with one topic on it.
Embedding	A list of numbers that captures what a passage *means*.	A 'meaning fingerprint'. Similar meanings have similar fingerprints.
Vector database	Storage that can quickly find passages with similar fingerprints.	A library shelved by meaning instead of alphabetically.
Re-ranking	A second, more careful model that re-reads the top results and re-orders them.	A librarian double-checking the books a search pulled.
Citation	A numbered reference [1] showing which passage supports a sentence.	A footnote in a report.
Guardrail	A safety rule the system must follow.	The rule 'never guess: say you don't know'.
CI (Continuous Integration)	Automatic checks that run every time the code is changed.	A checklist inspector at the factory door.

How one question travels through PolicyBot

Every question goes through the same six stages, and the whole journey usually takes about one to two seconds.



If nothing relevant is found in step 3-4, the assistant politely says it can only answer questions about company policies.

A real example

This exchange was recorded during testing. The numbers in square brackets are the sources; in the chat page they are clickable and open the policy at that exact section.

Question: How many unused PTO days can I carry over into next year, and by when must I use them?

PolicyBot: Employees may carry over a maximum of 5 unused PTO days into the next calendar year, and any carried-over days must be used by March 31 of that following year or they are forfeited[1][2].

Sources shown:
[1] **HR-002** Paid Time Off & Leave Policy - section *4. Paid Time Off (PTO) > 4.4 Carryover and year end*

[2] **HR-002** Paid Time Off & Leave Policy - section *10. Frequently Asked Questions*

Question: What is Lumora's current stock price?

PolicyBot: I can only answer questions about our company policies and procedures, and I couldn't find information about that in the policy documents.

This information is not in the policies, so PolicyBot declines instead of guessing.

What I did, step by step

I followed the assignment brief from start to finish. Each step below says what I did and why it matters.

Step 1 Understanding the assignment

What I did

I read the assignment document (a 7-page PDF) and turned it into a checklist: build a question-answering assistant over company policies, show sources, refuse off-topic questions, give it a web page, add automatic checks, measure accuracy and speed, and document the design decisions.

Why it matters

A clear checklist means nothing gets missed. Every item on it is covered by one of the steps below, and the final row of the checklist is in the last section.

Step 2 Setting up a clean workspace

What I did

I created a self-contained project folder with its own private set of software tools (a 'virtual environment'), so it doesn't interfere with anything else on the computer. I recorded the exact version of every tool, so anyone can rebuild the same setup later.

Why it matters

Recording exact versions makes the work **repeatable**. Another person, or the grader, gets the same results.

> **A real-world hiccup**
>
> The office network inspects secure internet traffic, which at first blocked the download of the AI models. I fixed this by telling the software to trust the computer's own security certificates, so it works both on the office network and anywhere else.

Step 3 Writing the company policy library

What I did

There was no real company data to use, so I wrote a realistic but fictional set of 13 policies for 'Lumora Systems': about 33,321 words, or roughly 67 printed pages. They cover vacation and leave, holidays, remote work, expenses, travel, IT security, acceptable use of computers, privacy, performance reviews, benefits, workplace conduct and onboarding. I wrote a single **fact sheet** first, listing every number (vacation days, spending limits, deadlines), and every document had to agree with it. I then had four AI writing assistants draft the documents at the same time.

Why it matters

Using made-up policies keeps real company information private while still being realistic. The fact sheet matters: if two documents disagreed (say, 15 vs 20 vacation days), nobody could judge whether an answer was right. I saved the documents in four file types (web page, formatted text, plain text and PDF) to show the system can read all of them.

ID	Policy	File type
FIN-001	Expense Reimbursement Policy	MD
FIN-002	Business Travel Policy	TXT
HR-001	Employee Handbook & Code of Conduct	MD
HR-002	Paid Time Off & Leave Policy	MD
HR-003	2026 Holiday Calendar & Holiday Policy	HTML
HR-004	Remote & Hybrid Work Policy	MD
HR-005	Performance Management & Compensation Policy	MD
HR-006	Employee Benefits Guide	MD
HR-007	Anti-Harassment, Equal Opportunity & Workplace Conduct Policy	MD
HR-008	Onboarding & Offboarding Procedures	PDF
LEG-001	Data Privacy & Protection Policy	MD
SEC-001	Information Security Policy	MD
SEC-002	IT Acceptable Use Policy	HTML

Step 4 Teaching the computer to read the documents

What I did

I built a reader for each file type. It strips out formatting and keeps the structure: each policy is split along its own headings (for example *'4. Paid Time Off > 4.1 PTO accrual by tenure'*), so every passage knows which section it came from.

Why it matters

Keeping the section names is what lets PolicyBot later say *exactly* where an answer came from.

Problem found and fixed

PDF files store text line by line as it appears on the page, so sentences were being cut in half at the end of each printed line and at page breaks. A clean-up step now joins those lines back into proper paragraphs. This one fix raised the 'sources point to the right passage' score from 96% to 100% in testing.

Step 5 Cutting the policies into bite-sized passages

What I did

I cut each section into passages of about 220 words (roughly half a page), with a small overlap between neighbouring passages so no sentence is lost at a boundary. I joined very short sections to the next one. The result: **382 passages**. Each passage is labelled with its policy name and section heading.

Why it matters

An AI answers best when it is given a few focused passages rather than whole documents. I chose half a page after testing three sizes (Step 12): smaller pieces lost context, and larger ones added noise.

Step 6 Making the passages searchable by meaning

What I did

I converted every passage into a 'meaning fingerprint' (an embedding) using a free AI model that runs on the computer itself. I stored the fingerprints in a small database (Chroma) kept inside the project folder.

Why it matters

Searching by meaning means a question like *"I left my work computer in a taxi"* still finds the rule about *"reporting lost or stolen devices"*, even though the words are different. Running the model locally means it is free, private and has no usage limits.

Step 7 Finding the best passages for each question

What I did

When a question arrives, it gets its own fingerprint, and the 20 closest passages are pulled from the database. A second, more careful model then re-reads those 20 alongside the question and keeps the **best 4**.

Why it matters

The first search is fast but rough; the second check is slower but sharper. In testing with casually worded questions, the double-check moved the correct passage to the top every time (it was in the top spot 94% of the time without it).

Step 8 Writing the answer

What I did

The 4 best passages, numbered [1] to [4], are handed to a large AI language model (openai/gpt-oss-120b, accessed through Groq's free service) with strict instructions: use **only** these passages, put the source number after every fact, keep it to about five sentences, and if the passages don't contain the answer, reply with a fixed polite refusal. If no AI key is set up, PolicyBot can still work in a simpler mode that quotes the most relevant sentences directly.

Why it matters

This is the 'open-book exam' idea. Because the AI may only use the passages it was given, its answers can be checked, and the numbered sources let the reader verify every sentence.

Problem found and fixed

The AI model originally planned for this project had been retired by the provider. I compared two available models on real policy questions and chose the stronger one. One of them writes source numbers in an unusual bracket style, so a small clean-up step converts them to the standard [1] form.

Step 9 Adding safety rules (guardrails)

What I did

I added several layers of protection, each catching what the one before might miss:

Why it matters

Together these make PolicyBot **trustworthy**: it won't invent policies, won't go off-topic, and never shows an answer without a source.

- **Off-topic filter:** if nothing in the policies resembles the question (e.g. *"What is the capital of France?"*), PolicyBot politely declines before the AI is even asked.
- **Refusal rule:** the AI itself is told to decline questions the policies don't cover, such as *"What is Lumora's stock price?"*
- **Sources required:** source numbers that don't match a real passage are removed. An answer with no sources is retried once, and if it still has none, it is replaced by the polite refusal.
- **Length limit:** answers are capped at a few sentences.
- **Input checks:** empty or extremely long questions are rejected with a clear message.
- **Error handling:** if the AI service is down, the user sees a friendly message, not technical errors.

Step 10 Building the website

What I did
I built a simple web application with three required parts: a **chat page** where you type a question and see the answer with clickable source cards (policy name, section, the exact quoted sentence and a link); a **/chat** address that other programs can send questions to; and a **/health** address that reports whether everything is running. A bonus **document viewer** lets you click a source and jump straight to that section of the policy.

Why it matters
This is how people actually use the assistant. The clickable sources make it easy to check any answer in seconds.

Step 11 Automatic quality checks (CI) and the run report

What I did
I wrote 20 automatic tests. They check that every file type is read correctly, that passages are cut consistently, that the safety rules work, and that the website answers real questions with sources and refuses off-topic ones. On GitHub, every time the code changes, a robot (GitHub Actions) installs everything from scratch, runs all the tests, starts the website, asks it a sample question, and publishes a **CI report** on the run page (test results, sample answer, document statistics and the latest accuracy numbers). The same report can be downloaded as a file.

Why it matters
Automatic checks catch mistakes before they reach users. If a future change breaks something, the run turns red immediately and the report shows what failed. When everything passes, the robot can also publish the new version to a public website.

Step 12 Measuring how good it is

What I did
I tested PolicyBot on a fixed set of questions and scored on accuracy, honesty about its sources, willingness to say 'I don't know', and speed.

Why it matters
Measuring, rather than assuming, shows whether the system can be trusted, and it guided several of the fixes described above.

How I tested it. I wrote 30 test questions in advance, each with a short 'correct answer' and the policy it should come from: 27 about the policies (vacation, holidays, travel, security, benefits, and so on) and 3 deliberately off-topic ones that PolicyBot should decline. Every expected fact was automatically confirmed to really exist in the policies. A second set of 16 casually worded questions (e.g. *"My flight to Tokyo is 13 hours, am I allowed to sit up front?"*) tested the search.

Who marked the answers. Exact facts and sources were checked automatically against the prepared answers. Whether an answer was fully backed by its sources was judged by a **different** AI model, so the writer was not marking its own homework.

AI-written answers (30 questions)

Measure	AI-written mode	Basic mode (no AI writer)
Answers backed by the documents	92.6%	100%
Sources pointing to the right passage	100%	100%
Answers containing all key facts	100%	88.9%
Policy questions answered	100%	100%
Off-topic questions declined	100%	66.7%
Typical answer time (half of answers faster)	1.2 seconds	0.7 seconds
Slow-case answer time (95% faster)	1.7 seconds	0.8 seconds

'Basic mode' is the fallback that quotes sentences directly when no AI key is available. It cannot recognise every off-topic question on its own, which is why the AI writer matters.

Fine-tuning the settings. I compared different settings to choose the defaults: how many passages to give the AI (1, 2, 4, 6 or 8), with or without the double-check, and passage sizes of about a quarter, half and a full page. **Four passages, with the double-check, at half a page each** was the smallest setup that found the needed facts for every test question.

Step 13 Publishing the work

What I did

I published all the code, the policy library, the tests, the evaluation results and the design documents on GitHub at **https://github.com/Valuoch/rag-policy-ai**, with a history of each change. The project includes the required documents: a README (how to set it up and run it), a design-and-evaluation document (why each choice was made, and the results), and an AI-tooling document (how AI tools helped build it).

Why it matters

This is what the grader reviews. Anyone can download the project and run it with three commands. Secret items, such as the AI service key, are deliberately kept out of the published code.

Assignment checklist

Requirement from the brief	Status	Where
Policy library: 5-20 documents, 30-120 pages	Done	13 documents, ~67 pages (Step 3)
Read PDF / web / text files; cut into passages	Done	Steps 4-5
Free embedding model + vector database	Done	Step 6
Top-k search with optional re-ranking	Done	Step 7
Answers with sources; refuse off-topic; length limit	Done	Steps 8-9
Web chat page, /chat and /health	Done	Step 10
Automatic checks on every change (GitHub Actions)	Done	Step 11
Evaluation: groundedness, citation accuracy, latency	Done	Step 12
Optional: comparison of settings (ablations)	Done	Step 12
README, design-and-evaluation, ai-tooling documents	Done	Step 13
Fixed settings for repeatable results	Done	Steps 2, 5, 12
Optional: public website (Render)	Ready, not yet switched on	See 'Next steps'
Demo video (5-10 minutes)	To do (presenter)	See 'Next steps'

Next steps

- **Put it online (optional):** connect the GitHub project to Render's free hosting, add the AI key there, and paste Render's 'deploy hook' link into GitHub. After that, every approved change goes live automatically.
- **Record the demo video:** show the chat page answering a few questions with sources, an off-topic question being declined, the CI report on GitHub, and the evaluation results.
- **Share the code:** make sure the GitHub account *quantic-grader* has access to the project.
- **Replace the AI key** after the project is graded, as good security practice.
- **Submit:** a single PDF containing the link to the GitHub project and the link to the demo video.

Appendix: how to run it yourself

On a computer with Python installed, in the project folder:

`pip install -r requirements.txt` (installs the tools)
`python -m rag.ingest` (reads the policies and builds the search index)
`python app.py` (starts the website at http://localhost:8000)

To enable AI-written answers, put a free Groq key in a file called `.env` (see `.env.example`). Without it, PolicyBot still runs in basic mode.